

Chapter 3: Command Line Parsing

argc / argv · switch statements · enums · getopt_long()

HOW TO READ THESE NOTES

- **NOT TESTED** boxes with a dashed gold border contain material you are **not responsible for** in this class. Included for completeness only.
- **AFTER MIDTERM** marks material covered later in the term.
- Everything else is fair game.

3.1 argc and argv (EECS 280 review)

Command line arguments

- **Command line arguments:** parameters passed to a program when it is run, used to dictate how the program behaves.
- Example command:

```
./game --level 25 --difficulty 10 --multiplayer
```

- `./game` runs an executable named `game` in the current directory.
- Arguments beginning with a dash are **command line options**. They specify features of the program and follow the executable name.
- Above: start the game at level 25, difficulty 10, multiplayer mode on.

The main() signature

```
int main(int argc, char* argv[]);
    -- or --
int main(int argc, char** argv);
```

- **argc** = the **number** of arguments passed in from the command line.
 - In the example there are **6**: `./game`, `--level`, `"25"`, `--difficulty`, `"10"`, `--multiplayer`.
- **argv** = an array of size `argc` holding **pointers to character arrays** (C-strings). Each C-string holds the contents of one argument.
- The **program name itself is included** and lives at `argv[0]`.

Layout of `argc = 6` and `argv` for `./game --level 25 --difficulty 10 --multiplayer`

<code>argv[0]</code>	→	<code>./game\0</code>
<code>argv[1]</code>	→	<code>--level\0</code>
<code>argv[2]</code>	→	<code>25\0</code>
<code>argv[3]</code>	→	<code>--difficulty\0</code>
<code>argv[4]</code>	→	<code>10\0</code>
<code>argv[5]</code>	→	<code>--multiplayer\0</code>
<code>argv[6]</code>	→	<code>NULL</code>

Everything in argv is a C-string

- All arguments, **including numbers**, are stored as C-strings.
- A **C-string** is a character array that ends with the null character `'\0'` (covered in detail in chapter 16).
- To get an integer out of `argv`, convert it with `atoi()`:
 - `atoi()` accepts a C-string and returns the integer it represents.
 - If the C-string cannot be interpreted as an integer, `atoi()` returns **0**.

- `atoi(argv[2])` returns `25` as an `int`.

REDIRECTION IS NOT PART OF ARGV / ARGV

```
./game --level 25 --difficulty 10 --multiplayer < input.txt > output.txt
```

- Arguments used for input/output redirection are excluded from both `argc` and `argv`.
- `argc` is still `6`; `argv` does **not** store "`<`", "`input.txt`", "`>`", or "`output.txt`".
- The values of `argc` and `argv` are identical to the non-redirected case.

3.2 Switch Statements and Enumerated Types

3.2.1 Switch statements

- A switch statement is an **alternative if-else structure for primitive types**. It is used by `getopt_long()`, so you will see it often.
- Advantages over a long if-else chain:
 - Neater / more readable.
 - Depending on the cases, the compiler may be able to **optimize** it for better performance.

```
switch (variable_to_test)           // value of this variable dictates what code runs
{
    case potential_value_1:         // if variable == potential_value_1, this case runs
    {
        /* code to run */
        break;                     // terminates the switch statement
    }
    case potential_value_2:
    {
        /* code to run */
        break;
    }
    ...                             // any number of cases
    default:                         // runs if no case matched (like the 'else' of if-else)
    {
        /* code to run */
    }
}
```

Rules

- You may have as many cases as you want, as long as:
 1. the variable in the switch **matches the data type** of the cases;
 2. the cases are **constant expressions that can be interpreted as integer values**.
- Syntax of a case: `case` keyword → the value to compare against → a colon.
- The switch variable is tested for **equality** against every case value.
- **Order of cases does not matter**. This differs from an if-else chain, where order may influence performance (in an if-else chain, list the most likely branches first).

break and fallthrough

- A matched case executes until a `break` is reached; then the switch terminates and execution continues after the switch.
- If **no break** is encountered at the end of a case, the program **falls through** into subsequent cases until a break is found.
- In other words: on a match, the program runs that case **and every case after it** until a break.

REMARK: [[FALLTHROUGH]]

- C++17 introduced the `[[fallthrough]]` attribute to explicitly mark an intentional fallthrough.
- Fallthroughs are usually unexpected and typically caused by a forgotten `break`, so some compilers warn about them.
- `[[fallthrough]]` suppresses that warning and is good style when fallthrough is actually intended.

The default case

- Optional, placed at the very end.
- Covers the condition where the switch variable matches none of the given cases.
- Needs **no break**, since no cases follow it (no risk of unintended fallthrough).

Curly braces per case

- Curly braces around a case body are **not always necessary**. Both versions below work:

```
case Day::Monday:
{
    std::cout << "It's Monday!\n";
    break;
}
case Day::Tuesday:
...
```

```
case Day::Monday:
    std::cout << "It's Monday!\n";
    break;
case Day::Tuesday:
...
```

- **But** the version without braces does **not** allow you to initialize variables inside a case. Only the left version below compiles:

```
case Day::Monday:
{
    int num_classes = 5;    // OK
    std::cout << "It's Monday!\n";
    break;
}
```

```
case Day::Monday:
    int num_classes = 5;    // ERROR
    std::cout << "It's Monday!\n";
    break;
```

WHY THE RIGHT VERSION FAILS

Without braces, the lifetime of a variable declared in one case **extends past that case** into the other cases (they share one scope). This does not compile. Adding curly braces to each case fixes it.

3.2.2 Enums AFTER MIDTERM**NOTE**

Enums are covered after the midterm, so do not worry if you have not seen them before then. They are still used with switch statements throughout the class.

The problem

- A switch only works on variables interpretable as an integer, so this is **illegal** (a `std::string` cannot be interpreted as an integer):

```
std::string day_of_week;
std::cin >> day_of_week;
switch (day_of_week)    // ERROR
{
    case "Monday": { std::cout << "It's Monday!\n"; break; }
    case "Tuesday": { std::cout << "It's Tuesday!\n"; break; }
    ...
    default:        { std::cout << "Invalid day!\n"; }
}
```

- One workaround is to use an integer per string (1 = Monday, 2 = Tuesday, ..., 7 = Sunday):

```
int32_t day_of_week;
std::cin >> day_of_week;
switch (day_of_week)
{
    case 1: { std::cout << "It's Monday!\n"; break; }
    case 2: { std::cout << "It's Tuesday!\n"; break; }
    ...
    default: { std::cout << "Invalid day!\n"; }
}
```

- This is complicated, messy, and error prone:
 - Does 1 mean Monday or Sunday? Should Sunday be 0 or 7?
 - Ambiguous, hard to read and maintain; other programmers may not know what the integers refer to.

The solution: enumerated types

- An **enum** is a user-defined type restricted to a certain set of values. It lets you use a condition's **name** in the switch instead of an integer.
- Under the hood, each name is associated with an integer, which is what the switch actually compares.

```
enum class <NAME OF ENUM> { ... VALUES OF ENUM ... }

enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

```
Day day_of_week = Day::Friday;
if (day_of_week == Day::Friday) {
    std::cout << "It's Friday!\n";
} // if
```

- `day_of_week` is an integer behind the scenes, not a string, but readability is preserved since real names appear in the code.

Rules for assigning enum values

- If the **first** enum value is not explicitly assigned, it gets the integer **0**.
- Any subsequent enum not explicitly assigned gets **one greater than the previous** enum's value.

EXAMPLE

For `enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };`
 Monday = 0, Tuesday = 1, Wednesday = 2, Thursday = 3, **Friday = 4**, Saturday = 5, Sunday = 6. Printing `Day::Friday` prints 4.

- Values are customizable with an initializer:

```
enum class Day { Monday = 1, Tuesday = 2, Wednesday = 3,
                Thursday = 4, Friday = 5, Saturday = 6, Sunday = 7 };

// identical, because each unassigned enum = previous + 1
enum class Day { Monday = 1, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
```

- **Two enum values may be assigned the same integer**, and enums can be compared just like integers.

Changing the underlying integer type

```
// stored as int8_t: takes 1 byte instead of 4
enum class Day : int8_t { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };

// a char is a 1-byte number behind the scenes, so chars work in switch cases too
enum class Day : char { Monday = 'm', Tuesday = 't', Wednesday = 'w',
                      Thursday = 'h', Friday = 'f', Saturday = 's', Sunday = 'u' };
```

A cleaner switch using an enum

```
enum class Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
Day day_of_week = ...;
switch (day_of_week)
{
    case Day::Monday: { std::cout << "It's Monday!\n"; break; }
    case Day::Tuesday: { std::cout << "It's Tuesday!\n"; break; }
    ...
    case Day::Sunday: { std::cout << "It's Sunday!\n"; break; }
    default:          { std::cout << "Invalid day!\n"; }
}
```

Grouping cases with fallthrough

```
enum class Suit { Diamonds, Hearts, Clubs, Spades };
Suit current_suit = ...;
switch (current_suit)
{
    case Suit::Diamonds: { [[fallthrough]]; }
    case Suit::Hearts:   { std::cout << "You got a red card!\n"; break; }
    case Suit::Clubs:    { [[fallthrough]]; }
    case Suit::Spades:   { std::cout << "You got a black card!\n"; break; }
    default:            { std::cout << "I don't know what your card is.\n"; }
}
```

- If the suit is Diamonds: the Diamonds case runs, there is no break, so it falls through and also runs Hearts (printing "You got a red card!"), then hits the break and exits without running Clubs or Spades.

WORKED EXAMPLE: TRACING FALLTHROUGH

```
1 enum class Weather { partly_cloudy, sunny, rainy, overcast };
2 Weather current_weather = ...;
3 switch (current_weather)
4 {
5     case Weather::partly_cloudy: {
7         std::cout << "There will be clouds in the sky.\n";
8         [[fallthrough]]; }
10    case Weather::sunny: {
12        std::cout << "The sun will be visible in the sky.\n";
13        break; }
15    case Weather::rainy: {
17        std::cout << "It will rain.\n";
18        [[fallthrough]]; }
20    case Weather::overcast: {
22        std::cout << "The sun will likely not be visible in the sky.\n";
23        break; }
25    default: {
27        std::cout << "Cannot identify weather\n"; }
29 }
```

Value	Runs lines	Output
partly_cloudy	5 → 13	There will be clouds in the sky. The sun will be visible in the sky.
sunny	10 → 13	The sun will be visible in the sky.
rainy	15 → 23	It will rain. The sun will likely not be visible in the sky.
overcast	20 → 23	The sun will likely not be visible in the sky.

enum class vs traditional enum

- Both forms technically define a new enum:

```
enum class <NAME OF ENUM> { ... VALUES OF ENUM ... }
enum <NAME OF ENUM> { ... VALUES OF ENUM ... }
```

- **Always include the `class` keyword.** Enum classes fix two bug-prone issues of traditional enums:

1. Traditional enums implicitly convert to integers, which may not be what the programmer wants. The compiler is not the programmer, so it should not decide on the programmer's behalf (same reasoning as the `explicit` keyword). Only enum classes prevent this implicit conversion.

```
enum Color { Red, Green, Blue, Yellow }; //
traditional

int main() {
    Color enum_value = Color::Blue;
    if (enum_value == 2) { // implicit
conversion, bad!
        std::cout << "blue" << std::endl;
    } // if
} // main()
```

```
enum class Color { Red, Green, Blue, Yellow };

int main() {
    Color enum_value = Color::Blue;
    if (enum_value == 2) { // compiler error,
good!
        std::cout << "blue" << std::endl;
    } // if
} // main()
```

```
enum class Color { Red, Green, Blue, Yellow };
int main() {
    Color enum_value = Color::Blue;
    if (static_cast<int32_t>(enum_value) == 2) { // okay, explicitly cast to integer
        std::cout << "blue" << std::endl;
    } // if
} // main()
```

2. Traditional enums export their names to the surrounding scope, causing clashes with other variable names. Names used in a traditional enum cannot be reused in the same scope or in another enum.

```
enum Color { Red, Green, Blue, Yellow };
enum Color2 { Red, Orange, Pink, Purple };
// not allowed, "Red" already used

int main() {
    enum Color3 { Brown, Indigo, Cyan, Teal };
    std::string Brown = "brown";
    // not allowed, "Brown" already used
} // main()
```

```
enum class Color { Red, Green, Blue, Yellow };
enum class Color2 { Red, Orange, Pink, Purple };
// okay

int main() {
    enum class Color3 { Brown, Indigo, Cyan,
Teal };
    std::string Brown = "brown";
    // okay
} // main()
```

3.3 Getopt Long

3.3.1 Why hand-parsing does not work

- Command: `./game --level 25 --difficulty 10 --multiplayer`
- **Tempting but wrong:** read the level from `argv[2]`, the difficulty from `argv[4]`, and check `argc` for multiplayer.
 - Fails if the options are given in a different order: `./game --multiplayer --level 25 --difficulty 10`
- **Better but still faulty:** walk the `argv` array and check for each option; if `--level` is seen, read the next index for the level number.
 - What if the user forgot the level number after `--level`?
 - What if `--multiplayer` could optionally be followed by a player count?

Short form vs long form options

- **Long form option:** the fully typed-out option, preceded by **two dashes** (`--level`, `--difficulty`).
- **Short form option:** an abbreviated form, preceded by a **single dash** (`-l`, `-d`, `-m`).
- In most programs short and long forms can be **mixed**, and short forms may be **combined**. These are all equivalent:

```
./game --level 25 --difficulty 10 --multiplayer
./game -l 25 -d 10 -m
./game --difficulty 10 -m -l 25
./game -md 10 --level 25
```

- Parsing all of this by hand is feasible for a few options but gets much harder as the option count grows, and the number of possible orderings makes error handling difficult.

3.3.2 Using getopt_long()

- GNU's `getopt_long()` lives in the `<getopt.h>` library and handles all of this for you. **This is what you will use in this class.**

```
1  #include <iostream>
2  #include <getopt.h>
3
4  // sets the difficulty of the game
5  set_game_difficulty(int32_t difficulty);
6  // sets the level of the game
7  set_game_level(int32_t level);
8  // turns multiplayer mode on
9  set_multiplayer();
10
11 int main(int argc, char** argv) {
12     int opt;
13     int opt_index = 0;
14     static struct option long_opts[] = {
15         { "difficulty", required_argument, nullptr, 'd' },
16         { "level",      required_argument, nullptr, 'l' },
17         { "multiplayer", no_argument,      nullptr, 'm' },
18         { nullptr, 0, nullptr, '\0' }
19     };
20
21     while ((opt = getopt_long(argc, argv, "d:l:m", long_opts, &opt_index)) != -1) {
22         switch (opt)
23         {
24             case 'd':
25             {
26                 // runs if 'd' or "difficulty" is specified on the command line
27                 std::cout << "difficulty set to " << atoi(optarg) << '\n';
28                 set_game_difficulty(atoi(optarg));
29                 break;
30             } // case 'd'
31             case 'l':
32             {
33                 std::cout << "level set to " << atoi(optarg) << '\n';
34                 set_game_level(atoi(optarg));
35                 break;
36             } // case 'l'
37             case 'm':
38             {
39                 std::cout << "multiplayer mode turned on\n";
40                 set_multiplayer();
41                 break;
42             } // case 'm'
43             default:
44             {
45                 std::cout << "unrecognized option" << std::endl;
46                 exit(1);
47             } // default
48         } // switch
49     } // while
50 } // main()
```

Part 1: the `long_opts[]` array

- `long_opts[]` is an array of `option` structs. Each valid option the user can type (`-d`, `-l`, ...) is its own `option` object.

```
struct option {
    const char* name;    // long form of the option, as a C-string
    int has_arg;         // whether the option must be followed by an argument
    int* flag;           // nullptr, or address of an int to set
    int val;            // short form char, or value to store into *flag
};
```

Member	Meaning
<code>name</code>	The long version of the option stored as a C-string, e.g. "difficulty" for <code>--difficulty</code> .
<code>has_arg</code>	An integer for whether the option needs a following argument. One of <code>no_argument</code> , <code>required_argument</code> , <code>optional_argument</code> .
<code>flag</code>	<code>nullptr</code> (to use short forms), or the address of an int variable to set.
<code>val</code>	The short form character (if <code>flag</code> is <code>nullptr</code>), or the value to write into <code>*flag</code> .

`has_arg`: the three option types

- **Required argument** — must be followed by an additional argument if included. `--level` must be followed by the level to go to (`--level 25`).
- **No argument** — must be run on its own. `--multiplayer` needs nothing after it.
- **Optional argument** — may or may not be followed by an additional argument; the argument is not necessary for the command to be valid. E.g. `--multiplayer` defaults to 2 players but `--multiplayer 3` customizes it.
 - **Important:** for optional arguments, the **argument** is the optional thing, not the option itself.

Value of <code>has_arg</code>	Underlying integer	Meaning
<code>no_argument</code>	0	Option takes nothing after it
<code>required_argument</code>	1	Option must be followed by an argument
<code>optional_argument</code>	2	Option may be followed by an argument

- These are enums, so you can type the word directly in the initializer list.
- **Specify the correct option type** for each option; mistakes here are difficult to track down.

`flag` and `val`: two behaviors

Behavior 1 — set an integer variable. Set `flag` to the address of an int and `val` to the value you want stored there.

```
static int num_players = 1;
int main(int argc, char** argv) {
    ...
    static struct option long_opts[] = {
        { "multiplayer", no_argument, &num_players, 2 },
        ...
    };
    ...
}
```

- `num_players` starts at 1; because `flag` is its address and `val` is 2, seeing `--multiplayer` changes `num_players` to 2.

Behavior 2 — specify a short form (what you will use in this class). Set `flag` to `nullptr` and `val` to the short-form character.

```
{ "difficulty", required_argument, nullptr, 'd' } // "difficulty" can be written as 'd'
```


THE TERMINATOR ENTRY IS MANDATORY

The last item of `long_opts[]` must be an option with **all members set to zero**. It tells `getopt_long` there are no more option choices remaining.

```
{ nullptr, 0, nullptr, '\0' } // or, equivalently: {0, 0, 0, 0}
```

Part 2: the while loop

```
while ((opt = getopt_long(argc, argv, "d:l:m", long_opts, &opt_index)) != -1) {
```

- `getopt_long` parses the command line and assigns the **short form** of the parameter into `opt`.
- It returns **-1** when there are no more command line arguments to read, ending the loop.

The short options string (3rd argument)

- Tells the function whether an option should expect an additional argument after it.
- Rules:
 - **Required argument** → short option followed by **one colon**.
 - **No argument** → short option followed by **nothing**.
 - **Optional argument** → short option followed by **two colons**.
- `"d:l:m"` means:
 - `d` (difficulty) has a required argument (followed by a colon).
 - `l` (level) has a required argument (followed by a colon).
 - `m` (multiplayer) has no argument (followed by nothing).
- **Order does not matter.** `"l:d:m"`, `"md:l:"`, `"ml:d:"`, and `"l:md:"` are all valid alternatives; only the colon placement matters.

Part 3: the switch statement and optarg

- Inside the while loop, a switch determines what the program does for each option encountered.
- Each **case is the short option** of a command; the case body implements the action taken.
- When an option has an additional argument, that argument is stored in a **global C-string called optarg**.
 - If `--level 25` is seen, `25` is stored in `optarg`.
 - `optarg` is a **C-string**: convert it to the correct type before use (e.g. `atoi(optarg)`).
- Add a **default case** for error handling. It runs when the encountered option matches none of the options defined in `long_opts[]`; in the example it prints a message and exits the program.

Part 4: optind and leftover (non-option) arguments

- `getopt_long` returns -1 when there are no more options, but there may be more arguments to read afterward:

```
./game --multiplayer --level 25 --difficulty 10 input.txt // input.txt is at argv[6]
./game -ml 25 -d 10 input.txt // input.txt is at argv[5]
```

- `input.txt` is not tied to an option, so `getopt_long` returns -1 before processing it. Finding its index by hand is not trivial, since it depends on how the options were written.
- `optind` is a global variable storing the **index of the next element in argv that needs to be processed**.
 - First command above: `optind = 6` after `getopt_long` finishes.
 - Second command: `optind = 5`.
- To check for leftover non-option terms: if `argc > optind` is true, there are additional terms you need to parse.

COMMON GETOPT_LONG GOTCHAS

- The colon goes in the **short options string** only, never in the `val` member of the option struct (`'r'`, not `"r:"`).
- Forgetting the all-zero terminator entry in `long_opts[]` is a bug.
- `long_opts[]` **can** legally be declared `static`.
- There is **no restriction** on specifying the same option more than once on the command line.
- An option declared `no_argument` must not be given an argument; an option declared `required_argument` must be given one, even when short forms are combined (in `-abg 12`, the `12` belongs to `g` only).

3.4 Boost Program Options

NOT TESTED — BONUS SECTION

Bonus material covering a resource you might see in industry. You are **not responsible for any of this** in this class. Included here only for completeness.

Overview

- In industry you may not use `getopt_long` because of its functional limitations. A popular alternative is `boost::program_options` from the boost library.
- For full functionality, consult the library's documentation.

```

1  #include <iostream>
3  #include <boost/program_options.hpp>
4  #include <boost/program_options/errors.hpp>
5  #include <boost/program_options/options_description.hpp>
6  #include <boost/program_options/parsers.hpp>
7  #include <boost/program_options/positional_options.hpp>
8  #include <boost/program_options/value_semantic.hpp>
9  #include <boost/program_options/variables_map.hpp>
11 static void greet_user(const std::string& name) {
12     std::cout << "Hello " << name << "!\n\n";
13 } // greet_user()
15 int main(int argc, char** argv) {
16     int32_t difficulty{};
17     std::vector<int32_t> levels_to_play;
18     std::string name;
19     bool is_multiplayer;
21     boost::program_options::options_description command_line_options{"Program Options"};
22     command_line_options.add_options()
23         ("difficulty,d", boost::program_options::value<int32_t>(&difficulty)->required(), "Game
difficulty")
24         ("level,l", boost::program_options::value<std::vector<int32_t>>(&levels_to_play)-
>multitoken()->composing()
25         ->zero_tokens(), "Levels to play")
26         ("name,n", boost::program_options::value<std::string>(&name)->default_value("Guest")-
>notifier(greet_user),
27         "Player name")
28         ("multiplayer,m", boost::program_options::bool_switch(&is_multiplayer)-
>default_value(false),
29         "Starts game in multiplayer mode")
30         ("help,h", "Prints out help message");
32     boost::program_options::variables_map options_map;
33     try {
34         boost::program_options::parsed_options parsed_result =
35             boost::program_options::parse_command_line(argc, argv, command_line_options);
36         boost::program_options::store(parsed_result, options_map);
37         if (options_map.count("help")) {
38             std::cout << command_line_options << '\n'; // prints options in pretty format
39             return 0;
40         } // if
41         boost::program_options::notify(options_map);
42         std::ostringstream oss;
43         for (int32_t level : levels_to_play) { oss << level << " "; }
44         std::cout << "Difficulty: " << difficulty << '\n';
45         std::cout << "Levels: " << oss.str() << '\n';
46         std::cout << "Name: " << name << '\n';
47         std::cout << "Multiplayer: " << std::boolalpha << is_multiplayer << '\n';
48     } // try
49     catch (boost::program_options::required_option& req) {
50         std::cout << "Missing required argument: " << req.get_option_name() << '\n';
51     } // catch
52     catch (boost::program_options::error& e) {
53         std::cout << "Error while parsing command line; Message=" << e.what() << '\n';
54     } // catch
55 } // main()

```

Compile and run:

```
g++ -std=c++1z -O3 -Wall -pedantic -pthread main.cpp -lboost_program_options -o game
./game --difficulty 10 --level 5 6 7 --name Dario --multiplayer
```

Output:

```
Hello Dario!
Difficulty: 10
Levels: 5 6 7
Name: Dario
Multiplayer: true
```

Registering options

- `options_description` (line 21) registers the command line options your program can handle.
- `add_options()` (line 22) defines the available options.
- Short options: add a comma and the letter after the long name (`"difficulty,d"` allows `--difficulty` and `-d`). A short option must be **one letter only**.
- Options with **two** parameters take no argument (like `no_argument` in `getopt_long`); options with **three** take an argument specified with `value<>` and its expected type.
- The string used to construct the description (`"Program Options"`) is the **caption**, printed when the object is printed. The caption is optional; without it only the list of valid commands is printed.

```
./game --help
```

Program Options:

```
-d [ --difficulty ] arg      Game difficulty
-l [ --level ] arg          Levels to play
-n [ --name ] arg (=Guest)  Player name
-m [ --multiplayer ]        Starts game in multiplayer mode
-h [ --help ]               Prints out help message
```

Feature summary

- `bool_switch()` — a switch argument that flips functionality on/off; providing `multiplayer` sets `is_multiplayer` to true.
- `notifier()` — invokes a separate function when the option is encountered ("Hello <NAME>!" is printed for `--name`).
- `required()` — the user must specify this option; failure raises an error when `notify()` is invoked (e.g. "Missing required argument: `--difficulty`").
- `default_value()` — value used if the option is not provided at all (`name` defaults to "Guest").
- `implicit_value()` — value used if the option is provided but with no value (the analogue of an optional argument in `getopt_long`), e.g. `value<std::string>(&name)->implicit_value("Guest")`.
- `multitoken()` — option can take multiple arguments (`--level 2 81`).
- `composing()` — option can be specified more than once (`--level 2 --level 81`).
- `zero_tokens()` — a multi-argument option may be followed by zero arguments (otherwise: "the required argument for option '`--level`' is missing").
- Methods can be **chained**: below, difficulty is 10 if not specified at all, and 15 if specified without a value.

```
( "difficulty,d",
  boost::program_options::value<int32_t>()->default_value(10)->implicit_value(15),
  "Game difficulty" )
```

Positional arguments

- **Positional arguments** are command line options not associated with a name, e.g. `./game input1.txt input2.txt input3.txt`.
- Step 1: add an entry with a name into `options_description` (a name is still needed, since it is how positional arguments are grouped). The value type should be a vector.

```
boost::program_options::options_description command_line_options;
command_line_options.add_options()
    ("input-files", boost::program_options::value<std::vector<std::string>>(), "Input files")
```

- Step 2: register it with a `positional_options_description` object.

```
boost::program_options::positional_options_description positional_options;
positional_options.add("input-files", -1); // -1 = unlimited
```

- The count controls how many positional arguments fall into that category: `-1` means unlimited; `add("input-files", 5)` means the next five, and nothing after. This lets you group positional arguments into categories:

```
boost::program_options::positional_options_description positional_options;
positional_options.add("player-names", 2); // two player names
positional_options.add("input-files", -1); // rest are input files
```

- Step 3: parse with the more advanced `command_line_parser()` instead of `parse_command_line()`, which is a simplified method that does **not** support positional arguments.

```
boost::program_options::variables_map options_map;
boost::program_options::parsed_options parsed_result =
    boost::program_options::command_line_parser(argc, argv)
        .options(command_line_options)
        .positional(positional_options)
        .run();
boost::program_options::store(parsed_result, options_map);
```

Full positional-argument example

```

11 int main(int argc, char** argv) {
12     int32_t difficulty{};
13     int32_t level{};
14     std::vector<std::string> player_names;
15     std::vector<std::string> input_files;
17     boost::program_options::options_description command_line_options{"Program Options"};
18     command_line_options.add_options()
19         ("difficulty,d", boost::program_options::value<int32_t>(&difficulty), "Game difficulty")
20         ("level,l", boost::program_options::value<int32_t>(&level), "Starting level")
21         ("player-names", boost::program_options::value<std::vector<std::string>>(&player_names),
22         "Player names")
23         ("input-files", boost::program_options::value<std::vector<std::string>>(&input_files),
24         "Input files")
25         ("help,h", "Prints out help message");
26     boost::program_options::positional_options_description positional_options;
27     positional_options.add("player-names", 2);
28     positional_options.add("input-files", -1);
29     boost::program_options::variables_map options_map;
30     try {
31         boost::program_options::parsed_options parsed_result =
32             boost::program_options::command_line_parser(argc, argv)
33                 .options(command_line_options).positional(positional_options).run();
34         boost::program_options::store(parsed_result, options_map);
35         if (options_map.count("help")) { std::cout << command_line_options << '\n'; return 0; }
36         boost::program_options::notify(options_map);
37         std::cout << "Difficulty: " << difficulty << '\n';
38         std::cout << "Level: " << level << '\n';
39         std::cout << "Player Names:\n";
40         for (const std::string& name : player_names) { std::cout << name << '\n'; }
41         std::cout << "Input Files:\n";
42         for (const std::string& file : input_files) { std::cout << file << '\n'; }
43     } // try
44     catch (boost::program_options::error& e) {
45         std::cout << "Error while parsing command line; Message=" << e.what() << '\n';
46     } // catch
47 } // main()

```

```
./game --difficulty 10 --level 25 Dario Paoluigi input1.txt input2.txt input3.txt
```

```

Difficulty: 10
Level: 25
Player Names: Dario Paoluigi
Input Files: input1.txt input2.txt input3.txt

```

Chapter 3 Quick Reference

Item	What it is
<code>argc</code>	Number of command line arguments, including the program name. Excludes redirection terms.
<code>argv</code>	Array of <code>argc</code> C-strings; <code>argv[0]</code> is the program name.
<code>atoi(cstr)</code>	Converts a C-string to an int; returns 0 if not interpretable as an integer.
<code>switch</code>	If-else alternative for integer-interpretable primitives; cases must be constant integer expressions.
<code>break</code> / <code>fallthrough</code>	No <code>break</code> means execution continues into following cases until a <code>break</code> is hit.
<code>enum class</code>	User-defined type restricted to a set of named values; first unassigned = 0, each next = previous + 1.
<code>struct option</code>	<code>{ name, has_arg, flag, val }</code> ; array must end with an all-zero entry.
short options string	colon = required arg, nothing = no arg, double colon = optional arg; order irrelevant.
<code>optarg</code>	Global C-string holding the argument supplied to the current option.
<code>optind</code>	Global index of the next unprocessed <code>argv</code> element; <code>argc > optind</code> means leftover terms exist.
return value of <code>getopt_long</code>	The short-form character of the option found, or -1 when no options remain.